ADVANCED DATA STRUCTURES AND DESIGN ENGINEERING

COURSE OUTCOME 4

Assignment details	
Subject	C PROGRAMMING
Student Name	AKSHAYA R S
Register Number	1924260209
Department	B.Tech Artificial Intelligence and Data Science
Course code	CSA0209
Github link	https://github.com/akshaya12258-droid/CSA0209-C-PROGRAMMING1924260209

TASK 1 – DATA DESIGN REVIEW (6 MARKS)

1.1 Diagnose the design (2 marks)

The original design stores one patient across five separate arrays. This is called a **structure of arrays** (SoA). It can work, but it creates several risks for a patient register:

Deficiency	What could go wrong at the health centre?
Parallel arrays	All arrays must use exactly the same index. If one array is shifted, inserted or sorted differently, a patient may be linked to another
Phone char[10]	A ten-digit phone number needs 11 bytes when stored as a C string: ten digits plus '\0'. char[10] can overflow if strcpy writes the te
One float for blood pressure	Blood pressure has two values, systolic and diastolic (for example 120 and 80). A single float cannot represent the pair clearly; par
Date stored as 8 characters	The format is unclear (DDMMYYYY, DDMMYY, etc.). Invalid dates such as 32132026 can be accepted, and text sorting may not g
No capacity check	When count reaches 100, writing name[100] and the other arrays is out of bounds. This causes undefined behaviour and may corru
Global arrays and count	Any function can modify the register or count. This makes errors difficult to locate, test and prevent.
Sorting is fragile	Sorting by age requires swapping the same index in every array. Missing one array silently corrupts

1.2 Repair the design (3 marks)

Use one Patient structure per person. Date and blood pressure are nested structures, so each concept has named fields. Pass a pointer to the record being filled. The caller owns the array and count, which avoids hidden global state.

C11 program / pseudocode

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#define MAX_PATIENTS 100

typedef struct {     int
day, month, year; } Date;

typedef struct {
int systolic;
int diastolic; }
BloodPressure;

typedef struct {
char name[50];
int age;
    char phone[16];       /* room for country code and terminating '\\0' */
BloodPressure bp;
    Date visit_date;
} Patient;

bool addPatient(Patient *p, const char *name, int age,
const char *phone, BloodPressure bp, Date date) {     if (p ==
NULL || name == NULL || phone == NULL) return false;     if (age
```

```
< 0 || age > 130) return false;     if (strlen(name) >= sizeof p-
>name ||
        strlen(phone) >= sizeof p->phone) return false;
if (bp.systolic <= 0 || bp.diastolic <= 0) return false;
if (date.day < 1 || date.day > 31 ||
        date.month < 1 || date.month > 12 || date.year < 1) return false;

    strcpy(p->name, name);       /* safe after length check */
p->age = age;     strcpy(p->phone, phone);     p->bp = bp;
p->visit_date = date;     return true; }

/* Example of capacity control at the call site:
   if (count < MAX_PATIENTS &&
       addPatient(&patients[count], name, age, phone, bp, date))
count++;
*/
```

Important: The capacity check belongs at the array-owning call site because addPatient receives one record pointer, not the entire array. In a production system, validate real calendar dates (including leap years), phone-number policy and clinically valid blood-pressure ranges.

1.3 Design defence (within 100 words) (1 mark)

A Patient structure keeps every field for one person together, so sorting or moving a patient moves one complete record rather than several parallel array elements. Functions can receive a pointer to one Patient, avoiding a full record copy and making updates clear. Nested Date and BloodPressure structures give meaningful names to related values and can be reused elsewhere. New fields can be added in one place, reducing the risk that arrays become misaligned.

Alternative concept and complexity analysis

Parallel arrays may be preferred when processing one field across a very large dataset (for example, calculating statistics for every age) because the selected field is contiguous and may use cache efficiently. They are also useful in column-oriented databases and vectorised numerical processing. For a small patient register with frequent whole-record operations, an array of structures (AoS) is easier and safer.

Measure	Parallel arrays (SoA)	Array of Patient structures (AoS)
Sort by age		ted field correctly. A record swap requires coordinated sw : O(n²); swap one complete Patient object.
Data moved per swap	Sum of the sizes of all fields, plus repeated string cop	ie . Exact amount d pends on field widths and implementa
Space	Sum of each array's element sizes × capacity. Paddi	g is n t shared across a record because fields are separa
Reliability	Index synchronisation must be maintained in every o	eration.One array element represents one complete atient.

sizeof(Patient) bytes logically copied for a temporary-based whole-struct swap; compil sizeof(Patient) × capacity. Padding may be inserted between members and at the end

For either representation, comparison-based sorting is O(n²) for selection sort or O(n log n) for an efficient comparison sort. A whole-structure swap is O(1) with respect to n, but copies sizeof(Patient) bytes. Padding is included in sizeof(Patient), so never calculate its size by simply adding member sizes unless the ABI layout is known.

TASK 2 – FROM REQUIREMENT TO NESTED STRUCTURES (8 MARKS)

2.1

Entity / structure	Fields	Relationship
Parcel	id, weight, booking date, sender, receiver, service type, charge	
Person / party	name, address	
Address	city, PIN code	Nested inside sender and receiver.
Date	day, month, year	Nested inside Parcel as booking date.
Service type	normal or express	

2.2 Define the structures (2 marks)

C11 program / pseudocode

```
#include <stdio.h>
#include <string.h>

#define MAX_PARCELS 100

typedef struct {     int
day, month, year; } Date;

typedef struct {
char city[30];
int pincode; }
Address;

typedef struct {
char name[50];
Address address; }
Party;

typedef enum {
NORMAL, EXPRESS }
ServiceType;

typedef struct {
int id;     float
weight;     Date
booked;
    Party sender;
    Party receiver;
ServiceType service;
float charge; } Parcel;

Parcel bookings[MAX_PARCELS]; /* array of structures for the day's bookings */
```

2.3 Design the operations (3 marks)

C11 program / pseudocode

```
PSEUDOCODE: SORT BY WEIGHT (ascending) for i = 0 to n
- 2     lightest = i     for j = i + 1 to n - 1
if parcel[j].weight < parcel[lightest].weight
lightest = j     if lightest != i
        swap the complete Parcel values at i and lightest
```

```
PSEUDOCODE: LIST BY RECEIVER CITY
read requested_city for each parcel
from index 0 to n - 1
    if receiver.address.city equals requested_city
print that parcel
```

Sorting uses selection sort: O(n²) comparisons and at most n−1 swaps. City search is O(n) because every parcel may need to be checked. String comparison should use strcmp, not ==.

2.4 Access paths (1 mark)

C11 program / pseudocode

```
/* (i) array and dot operators: third parcel, index 2 */
int pin1 = bookings[2].receiver.address.pincode;

/* (ii) pointer to the third parcel and arrow operator */
Parcel *p = &bookings[2];
int pin2 = p->receiver.address.pincode;
```

The arrow operator is shorthand for dereferencing a structure pointer and selecting a member: p->id is equivalent to (*p).id.

Complete the code – TODO 1 and TODO 2

C11 program / pseudocode

```
#include <stdio.h>

struct date { int day, month, year; }; struct
address { char city[20]; int pincode; }; struct
parcel {     int id;     float weight;
struct address to;     struct date booked; };

void print_parcel(const struct parcel *p)
{
    if (p == NULL) return;
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
p->id, p->weight, p->to.city, p->to.pincode,            p-
>booked.day, p->booked.month, p->booked.year); }

void sort_by_weight(struct parcel p[], int n)
{
    for (int i = 0; i < n - 1; i++) {
int lightest = i;
        for (int j = i + 1; j < n; j++) {
if (p[j].weight < p[lightest].weight)
lightest = j;         }
        if (lightest != i) {
struct parcel temp = p[i];
p[i] = p[lightest];
p[lightest] = temp;         }
    }
}

int main(void)
{
    struct parcel list[3] = {         {501, 4.5f,
{"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
};
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) print_parcel(&list[i]);
return 0; }
```

Expected output

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
```

A swap assigns one complete struct parcel into a temporary and performs two whole-structure assignments back into the array. At the source level, three assignments participate in the swap; the temporary and each destination assignment involve sizeof(struct parcel) bytes of object representation. A common answer to “bytes copied in each swap” is **3 × sizeof(struct parcel)** bytes of struct assignment traffic (one read into temp and two writes), while actual machine traffic can differ because the compiler may optimise.

Alternative: flat structure and space comparison

A flat design places receiver city and PIN, and booking day/month/year, directly in Parcel. Its access expression becomes parcel[2].receiver_city and parcel[2].receiver_pincode; there is no reusable Address member type. Nested structures improve conceptual grouping and allow Address to be reused for sender, receiver and other records. The flat version may have the same total member storage, but padding can differ because member order and alignment differ. Compare with sizeof on the target compiler; do not assume nested structures always use more bytes.

TASK 3 – UNIONS AND MEMORY LAYOUT (9 MARKS)

3.1 Structure or union? (3 marks)

C11 program / pseudocode

```
/* Structure design: all fields have storage simultaneously */
struct reading_all {     float celsius;     int percent;
char status[8]; };

/* Tagged-union design: exactly one alternative is active */
enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
enum kind type;
union {          float
celsius;          int
percent;          char
status[8];      } value;
};
```

The station sends exactly one value per packet, so the tagged union is the better design. A normal structure reserves space for every member even when only one is meaningful. A union overlays its members in the same storage; the tag records which member is currently valid. The tag and union must always be updated together.

3.2 Predict and verify sizeof (2 marks)

On a typical GCC x86-64 ABI, float and int are 4 bytes and char[8] is 8 bytes. The union must be large enough for its largest member (8 bytes) and aligned for its strictest member (commonly 4 bytes). The enum tag is commonly 4 bytes. Thus the tagged structure is commonly 4 + 8 = 12 bytes, with no extra padding required between these two members under that ABI. The all-fields structure is commonly 4 + 4 + 8 = 16 bytes, possibly with tail padding to its alignment (typically still 16).

Size check (add to a small test program)

```
printf("sizeof all-fields structure = %zu\n", sizeof(struct reading_all));
printf("sizeof union = %zu\n", sizeof(((struct reading *)0)->value));
printf("sizeof tagged reading = %zu\n", sizeof(struct reading));
```

The assignment's expected union size 8 and tagged structure size 12 are consistent with the typical ABI described above. Verify on the target system because enum size, alignment and padding are implementation-dependent.

3.3 Complete TODO 1 and executable program (3 marks)

C11 program / pseudocode

```
#include <stdio.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
enum kind type;
union {          float
celsius;          int
percent;          char
status[8];      } value;
};

void show(const struct reading *r)
{     if (r == NULL) return;
switch (r->type) {          case
TEMPERATURE:
            printf("Temp %.1f C\n", r->value.celsius);
break;          case HUMIDITY:
            printf("Humidity %d %%\n", r->value.percent);
break;          case STATUS:             printf("Status
%s\n", r->value.status);             break;
default:
            printf("Invalid packet tag\n");
    }
}

int main(void) {     struct reading log[3]
= {          {TEMPERATURE, {.celsius =
31.5f}},
        {HUMIDITY, {.percent = 68}},
```

```
        {STATUS, {.status = "OK"}}     };     printf("sizeof
union = %zu, sizeof struct reading = %zu\n",            sizeof
log[0].value, sizeof log[0]);     for (int i = 0; i < 3; i++)
show(&log[i]);     return 0; }
```

Expected output

```
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68 %
Status OK
```

3.4 Misuse experiment (1 mark)

C11 program / pseudocode

```
struct reading r;
r.type = TEMPERATURE;
r.value.celsius = 25.0f;
printf("%d\n", r.value.percent); /* deliberate misuse */
```

This reads a union member other than the one most recently stored. In C, reading an inactive union member is not a portable way to convert the value. The result is implementation-dependent/unspecified in practical terms and may look like an unrelated integer representation of the float. Do not predict one universal numeric output. The tag must be checked before reading the union.

Alternative concept and complexity analysis

Design	Read a packet	Space and trade-off
Tagged union array	O(1): inspect tag, then read the active m	ember.
Three separate arrays	O(1) if caller already knows the array an	index; otherwi type-to-array routing is needed.
malloc + void pointer	Dereference after casting to the correct p	ointer type; usually O(1).

Per packet sizeof(tagged struct) typically 12 bytes here. Type i explicit and storage is shared

Each value uses its own array's elen

size, but a shared packet sequence/order needs extra Each packet needs allocation

metadata/allocator overhead and a type tag somewhere. More fra

For thousands of compact, mixed-type packets, a tagged union array is usually simplest and memory-efficient. Dynamic allocation is justified only when packet sizes or lifetimes vary enough to outweigh allocator overhead.

TASK 4 – STRUCTURE FORENSICS (7 MARKS)

4.1 Locate the defects (2 marks)

Defect 1: addInterest receives struct account by value. C copies the account into parameter a, so the function changes only its local copy. The original list element remains unchanged. Defect 2: p is a pointer, so p.number and p.balance are invalid member access expressions. Use p->number and p->balance (or (*p).number).

4.2 Construct address evidence (2 marks)

C11 program / pseudocode

```
#include <stdio.h> struct account { int
number; float balance; };

void inspect(struct account a) {     printf("address of
parameter copy = %p\n", (void *)&a); } int main(void) {
struct account item = {101, 5000.0f};     printf("address of
original      = %p\n", (void *)&item);     inspect(item);
return 0; }
```

The two printed addresses should be different because the parameter is a separate object. Actual address values vary on every run. The address comparison is evidence of pass-by-value; it is not necessary to rely on a particular hexadecimal address.

4.3 Repair using structure pointer (2 marks)

C11 program / pseudocode

```
#include <stdio.h>

struct account { int number; float balance; };

void addInterest(struct account *a, float rate)
{     if (a == NULL || rate < 0.0f) return;
    a->balance = a->balance + a->balance * rate / 100.0f;
}

int main(void) {     struct account list[3] = {
{101, 5000.0f}, {102, 12000.0f}, {103, 800.0f}     };
    struct account *p = list;
for (int i = 0; i < 3; i++)
addInterest(&list[i], 4.0f);

    for (int i = 0; i < 3; i++)
        printf("%d %.2f\n", p[i].number, p[i].balance);
return 0; }
```

Expected output

```
101 5200.00
102 12480.00
103 832.00
```

4.4 Prevent recurrence (1 mark)

Parameter form	Use when	Effect
struct account a (value)	Small structures or when the function must work on a p	ivate copy.
struct account *a	Function must modify caller's object or avoid copying a	arge tructure.
const struct account *a	Function only reads a structure, especially a large one.	

Copies the structure; changes do not affect caller.

Passes an address; use ->; caller's object can change.

Avoids structure copy and prevents modification through this pointer.

Alternative: return modified value and complexity

C11 program / pseudocode

```
struct account addInterestValue(struct account a, float rate)
{
```

```
    a.balance += a.balance * rate / 100.0f;
return a; }

/* call */ for (int i = 0;
i < 3; i++)
    list[i] = addInterestValue(list[i], 4.0f);
```

The value-return design is clear and useful for small immutable-style transformations. Conceptually, the input parameter copy and returned value each involve sizeof(struct account) bytes of value transfer; a compiler may use registers or a hidden return slot and optimise copies away. Pointer passing transfers one pointer (typically 8 bytes on a 64-bit system) and modifies the original. Both use O(1) time per account and O(n) for n accounts. The pointer version uses little parameter storage; the value version needs storage for the parameter/result, though exact stack use is ABIand optimisation-dependent.

TASK 5 – STORAGE CLASSES: THE SAME COUNTER FOUR WAYS (7 MARKS)

5.1 Four implementations (3 marks)

(a) Automatic local: initialized on every call; it forgets the previous value.

C11 program / pseudocode

```
int auto_token(void) {     int token = 1000;
/* automatic local */     return ++token; }
```

(b) Static local: retains its value between calls, but its name is visible only inside the function.

C11 program / pseudocode

```
int next_token(void) {     static int token = 1000; /* initialized once, zero-
initialized if omitted */     return ++token; }
```

(c) Global shared across two files: define the object once in one .c file and declare it with extern in another. Do not put a second definition in the header.

C11 program / pseudocode

```
/* counter.c */ int shared_token = 1000; int
issue_shared_token(void) { return ++shared_token; }

/* main.c */ #include
<stdio.h> extern int
shared_token; int
issue_shared_token(void); int
main(void) {
    printf("%d %d %d\n", issue_shared_token(),
issue_shared_token(), issue_shared_token());     return
0; }
```

Compile both source files together: `gcc -std=c11 -Wall -Wextra main.c counter.c -o mess`. The extern declaration refers to the one definition in counter.c.

(d) Register loop variable: register is a storage-class hint for a local variable, not a persistent counter. It may be used for a loop index; modern compilers choose registers automatically.

C11 program / pseudocode

```
#include <stdio.h>
int main(void) {
    for (register int i = 0; i < 3; i++)
printf("token display row %d\n", i);
return 0; }
```

The register variable in this example is only the loop index. It does not itself preserve token numbers. Taking its address with &i; is not allowed by the C language.

Complete TODO 1 and expected output

The completed next_token function is the static-local implementation above. The assignment's program prints:

Expected output

```
static: 1001 1002 1003
auto : 1001 1001 1001
```

5.2 Two independent counters

With automatic variables, create two independent functions or pass a counter object; a local automatic value still resets on every call. With static locals, create separate functions such as next_veg() and next_nonveg(), each with its own static int; this is a small change but duplicates logic. With globals, define veg_token and nonveg_token and declare both extern in the other file; add a selector or two functions. With a register loop variable, the loop index does not hold token state, so the actual counter must be stored elsewhere. Best maintainable solution: keep two ordinary int values in the caller and pass the selected counter by pointer.

Kind	Scope / visibility	Lifetime	Initial value	Main risk
Automatic local	Block in which declared	Until block/function ends	Indeterminate if not initializ	ed
Static local	Block only	Entire program execution	Zero if not explicitly initializ	ed
Global (extern declaratio	n)File scope; extern makes definition	usable across translation unitsEntire program execution	Zero if not explicitly initializ	ed
register local	Block in which declared	Until block/function ends	Indeterminate if not initializ	ed

5.3 Compare storage classes (2 marks)

Resets each call; using uninitialized value is unsafe. Hidden shared state; calls affect one another. Any code with access can modify it; name collisions a

Address cannot be taken; register is only a hint,

Unspecified argument evaluation order: In C, the order of evaluating function arguments is not specified. Therefore printf("%d %d %d", next_token(), next_token(), next_token()) may call next_token in a different order from left to right. Since each call modifies the same counter, the values can appear in an unexpected order. Use separate statements to guarantee the sequence.

Alternative: caller-owned pointer counter and complexity

C11 program / pseudocode

```
int next_token(int *counter)
{
    if (counter == NULL) return -1;
++(*counter);     return *counter;
}

/* caller */ int veg = 1000;
int nonveg = 1000; int v =
next_token(&veg); int nv =
next_token(&nonveg);
```

Each call is O(1), just like static and global versions. The counter occupies sizeof(int) in the caller's storage and lives as long as that caller-owned object. Passing a pointer adds only pointer-sized parameter storage. Two or more independent counters are easy: pass &veg; or &nonveg.; This design makes ownership explicit and avoids hidden global/static state.

TASK 6 – DESIGN CHALLENGE: COLLEGE CANTEEN INVENTORY AND BILLING (13 MARKS)

Stage 1 – Interpret the problem (2 marks)

Entities: **Item** (common ID, name, price, stock and kind-specific data); **Packaged item** (expiry date); **Cooked item** (preparation time); **Combo meal** (up to three component item IDs and quantities); **Sale line** (item ID, quantity, line revenue); and **daily summary** (sales lines and total revenue).

Rules and testable requirements:

ID	Testable requirement
R1	The program stores a unique positive item ID, non-empty name, non-negative price and non-negative stock.
R2	A packaged item stores a valid expiry date; a cooked item stores preparation time; a combo stores 1–3 component references
R3	A sale is accepted only if the item exists, quantity is positive and sufficient stock is available.
R4	On an accepted sale, stock decreases by the sold quantity and revenue increases by price × quantity.
R5	An item is warned about when its expiry date is today, tomorrow or the day after tomorrow (within two days). Expired items are
R6	A rejected sale does not change stock or revenue.
R7	The day's summary is sorted by line revenue in descending order.

For a small demonstration, combo components are recorded as IDs and quantities; combo stock is maintained as its own sellable stock. A real canteen system would also decrement component stocks atomically when a combo is sold.

Stage 2 – Propose the data model (2 marks)

Use a Date nested structure, a tagged union for the three item kinds, an array of Item structures for inventory, and an array of SaleLine structures for the day's sales. The tag must match the active union member.

C11 program / pseudocode

```
typedef struct { int day, month, year; } Date;
typedef enum { PACKAGED, COOKED, COMBO } ItemKind;

typedef struct { Date expiry; } PackagedData;
typedef struct { int hour, minute; } CookedData;
typedef struct {     int component_id[3];
int component_qty[3];     int component_count; }
ComboData;

typedef struct {     int id;
char name[40];     double
price;     int stock;
ItemKind kind;     union {
PackagedData packaged;
        CookedData cooked;
        ComboData combo;
    } detail;
} Item;

typedef struct {
int item_id;
int quantity;
double revenue; }
SaleLine;
```

Stage 3 – Critical design decisions (2 marks)

Stock is updated only after checking item existence, positive quantity and sufficient stock. The sale function first validates all conditions, then changes stock and appends a SaleLine; a failed sale changes nothing. Expiry uses a Date structure and a date-difference helper rather than comparing date strings. The sample uses a fixed inventory array and a fixed sales array, so no global variables are necessary: arrays and counts are owned by main and passed to functions. A static/global counter is unnecessary unless the program needs a persistent shared transaction number.

For reliable production use, validate actual calendar dates, use integer paise rather than floating-point currency, persist records to a file/database, and handle inventory updates transactionally.

Stage 4 – Structures, prototypes and pseudocode (4 marks)

C11 program / pseudocode

```
int find_item(const Item items[], int count, int id); int
record_sale(Item items[], int item_count,                SaleLine
sales[], int *sale_count,                int sale_capacity, int
item_id, int quantity); int days_from_civil(Date d); /* converts a
valid date to day number */ void warn_expiring(const Item items[], int
count, Date today); void sort_sales_by_revenue(SaleLine sales[], int
count);

PSEUDOCODE: RECORD SALE find the item by ID if item
not found: reject if quantity <= 0: reject if
quantity > available stock: reject if sales array
is full: reject calculate line revenue = item price
* quantity decrease item stock by quantity append
item ID, quantity and revenue to sales array accept
sale

PSEUDOCODE: EXPIRY WARNING convert today's
date to an integer day number for each
packaged item
    convert expiry date to integer day number
difference = expiry - today     if difference <
0: print EXPIRED     else if difference <= 2:
print EXPIRING SOON
```

Complete working C11 program (3 marks)

C11 program / pseudocode

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#define MAX_ITEMS 10
#define MAX_SALES 50

typedef struct { int day, month, year; } Date;
typedef enum { PACKAGED, COOKED, COMBO } ItemKind;
typedef struct { Date expiry; } PackagedData;
typedef struct { int hour, minute; } CookedData;
typedef struct {     int component_id[3];     int
component_qty[3];     int component_count; }
ComboData;

typedef struct {     int id;
char name[40];     double
price;     int stock;
ItemKind kind;     union {
PackagedData packaged;
        CookedData cooked;
        ComboData combo;
    } detail;
} Item;

typedef struct {
int item_id;
int quantity;
double revenue; }
SaleLine;

/* Howard Hinnant's civil-date conversion, for valid Gregorian dates. */
static int days_from_civil(Date d) {
    int y = d.year;
int m = d.month;
int day = d.day;     y
-= (m <= 2);     int
era = (y >= 0 ? y : y
- 399) / 400;
unsigned yoe =
(unsigned)(y - era *
400);     unsigned doy
= (153U * (unsigned)(m
+ (m > 2 ? -3 : 9)) +
2U) / 5U
+ (unsigned)day - 1U;
```

```
unsigned doe = yoe *
365U + yoe / 4U - yoe
/ 100U + doy;
return era * 146097 +
(int)doe; }

static int valid_date(Date d) {     static const int mdays[] =
{31,28,31,30,31,30,31,31,30,31,30,31};     if (d.year < 1 || d.month < 1 ||
d.month > 12 || d.day < 1) return 0;     int maxday = mdays[d.month - 1];
int leap = (d.year % 400 == 0) || (d.year % 4 == 0 && d.year % 100 != 0);
if (d.month == 2 && leap) maxday = 29;     return d.day <= maxday; }

int find_item(const Item items[], int count, int id)
{     for (int i = 0; i < count; i++)         if
(items[i].id == id) return i;     return -1; }

int record_sale(Item items[], int item_count, SaleLine sales[],
int *sale_count, int sale_capacity,                 int item_id, int
quantity) {     if (sale_count == NULL || quantity <= 0 || *sale_count >=
sale_capacity)         return 0;
    int index = find_item(items, item_count, item_id);     if
(index < 0 || quantity > items[index].stock) return 0;

    /* Validate first; update inventory and sales only after all checks pass. */
double amount = items[index].price * quantity;     items[index].stock -=
quantity;     sales[*sale_count].item_id = item_id;
sales[*sale_count].quantity = quantity;     sales[*sale_count].revenue = amount;
++(*sale_count);     return 1; }

void warn_expiring(const Item items[], int count, Date today)
{
    if (!valid_date(today)) {
printf("Invalid current date\n");
return;     }
    int today_no = days_from_civil(today);     for (int i = 0; i <
count; i++) {         if (items[i].kind == PACKAGED &&
valid_date(items[i].detail.packaged.expiry)) {             int diff =
days_from_civil(items[i].detail.packaged.expiry)
- today_no;             if (diff < 0)
                printf("EXPIRED: %s\n", items[i].name);
else if (diff <= 2)                 printf("EXPIRING SOON: %s
(within %d day(s))\n",                         items[i].name,
diff);         }
    }
}

void sort_sales_by_revenue(SaleLine sales[], int count)
{
    for (int i = 0; i < count - 1; i++) {
int best = i;
        for (int j = i + 1; j < count; j++)             if
(sales[j].revenue > sales[best].revenue) best = j;         if
(best != i) {             SaleLine temp = sales[i];
sales[i] = sales[best];             sales[best] = temp;         }
    }
}

int main(void)
{
    Date today = {11, 11, 2026};
    Item items[MAX_ITEMS] = {
        {1, "Milk", 30.00, 10, PACKAGED, {.packaged = {{12,11,2026}}}},
        {2, "Sandwich", 50.00, 5, COOKED, {.cooked = {9,30}}},
        {3, "Veg Combo", 80.00, 4, COMBO,
         {.combo = {{2,2,0}, {1,1,0}, 2}}}
};
    int item_count = 3;
SaleLine sales[MAX_SALES];
int sale_count = 0;

    printf("SALE 1: ");
    if (record_sale(items, item_count, sales, &sale_count,
MAX_SALES, 1, 2))         printf("Milk x2 accepted\n");
else         printf("rejected\n");

    printf("SALE 2: ");
    if (record_sale(items, item_count, sales, &sale_count,
MAX_SALES, 2, 7))         printf("Sandwich x7
accepted\n");     else         printf("rejected -
insufficient stock\n");
```

```
    printf("SALE 3: ");
    if (record_sale(items, item_count, sales, &sale_count,
MAX_SALES, 3, 1))         printf("Veg Combo x1
accepted\n");     else          printf("rejected\n");

    printf("\nExpiry warnings for 11/11/2026:\n");
warn_expiring(items, item_count, today);

    sort_sales_by_revenue(sales, sale_count);
printf("\nDaily summary (highest revenue first):\n");
for (int i = 0; i < sale_count; i++) {
        int idx = find_item(items, item_count, sales[i].item_id);
printf("%s | qty %d | Rs %.2f\n",
               items[idx].name, sales[i].quantity, sales[i].revenue);
}
    printf("Remaining stock: Milk=%d, Sandwich=%d, Veg Combo=%d\n",
items[0].stock, items[1].stock, items[2].stock);     return 0; }
```

Expected output

```
SALE 1: Milk x2 accepted
SALE 2: rejected - insufficient stock
SALE 3: Veg Combo x1 accepted

Expiry warnings for 11/11/2026:
EXPIRING SOON: Milk (within 1 day(s))

Daily summary (highest revenue first):
Veg Combo | qty 1 | Rs 80.00
Milk | qty 2 | Rs 60.00
Remaining stock: Milk=8, Sandwich=5, Veg Combo=3
```

The example's packaged milk expires on 12/11/2026, one day after the sample date, so it triggers the two-day warning. Sandwich stock remains 5 because the attempted quantity 7 is rejected without changing inventory. Revenue is sorted descending: Rs 80, then Rs 60.

Operation	Array of structures	Singly linked list
Add item	O(1) at end if capacity is available; O(n) if insertion re	quires shifting.
Remove item		t (or O(1) after lookup if order need not be preserved and last element is move
Search by ID	O(n) linear search; O(log n) only if sorted and binary-	searchable in an array.O(n) linear search.
Memory	Contiguous Item records; no per-node link pointer; po	ssible unus d fix d capacity.

Alternative concept and complexity analysis: linked list versus array

O(1) at head or tail when a tail pointer is maintained; O(n) to find a sorted insertion poin O(n) to find item and predecessor; O(1) unlink once predecessor is known.

Each node stores Item plus one next pointer and allocator overhead; memory is allocate

A linked list is useful when the number of items changes frequently and stable node addresses are valuable. An array is simpler, cache-friendly and usually faster for a small fixed canteen inventory. In either design, the sale lookup remains O(n) unless an additional index (for example, a hash table) is used.

.